

PROYECTO TITEC, CICLO 2026

Consolidado de Contratos de Backend

entre microservicios de la plataforma de eventos

Resumen por microservicio: tema de cada contrato y cómo está definido

Documento	Consolidado de contratos de backend TITEC 2026
Estado	Emitido para revisión de los grupos
Alcance	9 microservicios: Auth, Catálogo, Check-in, Entradas / Inventario, Notificaciones, Pagos, Panel Organizador, Promociones y Reseñas
Contratos revisados	33 acuerdos de backend de los 9 microservicios, con coincidencias entre equipos
Naturaleza del contenido	Contratos de backend entre microservicios: propósito, equipos involucrados, operación, endpoint o canal de comunicación, datos y SLA, y su referencia funcional
Nivel de detalle	Resumen ejecutivo por contrato; no se replican request/response completos

Documento de trabajo. Cada sección corresponde a un microservicio y a su equipo responsable; dentro de cada una se listan los contratos del equipo indicando qué expone, quién lo consume y a través de qué backend.

1. Índice de microservicios y contratos

Vista rápida de los 9 microservicios y de los acuerdos que cada equipo administra.

Microservicio	Contratos y contrapartes	Nº
Auth (Autenticación)	- Notificaciones <-> Auth	1
Catálogo de Eventos (Equipo 7)	- Panel Organizador <-> Catálogo de Eventos - Reseñas <-> Catálogo de Eventos - Entradas / Inventario <-> Catálogo de Eventos	3
Check-in	- Panel organizador <-> Check-in - Entradas / Inventario <-> Check-in - Reseñas <-> Check-in	3
Entradas / Inventario	- Entradas / Inventario <-> Autenticación (Auth) - Entradas / Inventario <-> Catálogo de Eventos - Entradas / Inventario <-> Check-in - Entradas / Inventario <-> Pagos - Entradas / Inventario <-> Promociones - Catálogo de Eventos <-> Entradas / Inventario - Entradas <-> Notificaciones - Pagos <-> Entradas / Inventario - Entradas / Inventario <-> Check-in - Entradas / Inventario <-> Panel Organizador	10
Notificaciones	- Entradas <-> Notificaciones - Panel Organizador <-> Notificaciones - Notificaciones <-> Auth	3
Pagos	- Entradas / Inventario <-> Pagos - Promociones <-> Pagos	2
Panel Organizador	- Auth <-> Panel Organizador - Panel Organizador <-> Catálogo de Eventos - Panel Organizador <-> Check-in - Panel Organizador <-> Entradas / Inventario - Panel Organizador <-> Notificaciones - Panel Organizador <-> Promociones	6
Promociones	- Entradas <-> Promociones - Promociones <-> Pagos	2
Reseñas	- Reseñas <-> Check-in	3

Microservicio	Contratos y contrapartes	Nº
	- Reseñas <-> Catálogo de Eventos - Catálogo de Eventos <-> Reseñas	

2. Tabla de dependencias entre grupos

Visión general de cómo se conectan los microservicios según los contratos de backend. Los números del diagrama corresponden a los flujos de la tabla siguiente.

Flujos de integración

#	Flujo	Quién expone	Quién consume	Backend
1	Validación de sesión (introspección del JWT)	Autenticación (Auth)	Entradas / Inventario	REST – GET /internal/validar-sesion
2	Datos maestros y ciclo de vida del evento; publicación de stock	Panel Organizador	Entradas / Inventario	RabbitMQ (panel.evento.entradas.v1 / entradas.evento.stock.v1) + REST
3	Procesar compra de entradas	Entradas / Inventario	Catálogo de Eventos	REST – POST /api/v1/entradas/procesar-compra
4	Iniciar pago, consultar estado y anular o reembolsar	Pagos	Entradas / Inventario	REST – POST /api/v1/pagos/transacciones (+ flujo asíncrono)
5	Envío del QR de acceso y consulta de datos de la entrada	Entradas / Inventario	Check-in	REST – POST /api/v1/checkin/tickets, GET /api/v1/entradas/{id_ticket}
6	Notificación de entrada emitida	Entradas / Inventario	Notificaciones	RabbitMQ – exchange entradas.events
7	Datos del evento para mostrar junto a la reseña	Catálogo de Eventos	Reseñas	REST – GET /eventos/{eventoId}
8	Verificación de asistencia al evento	Check-in	Reseñas	REST – POST /checkin/asistencia
9	Confirmación del pago con descuento	Pagos	Promociones	REST – GET /api/v1/pagos, evento pago.completado
10	Validación de cupón y cálculo del descuento	Promociones	Entradas / Inventario	REST – POST /promociones/validar/{nombre_codigo}
11	Cobro directo con tarjeta	Pagos	Pasarela de pago (Stripe)	url_checkout (integración externa)
12	Eventos próximos, búsqueda con filtros y detalle de evento	Panel Organizador	Catálogo de Eventos	REST - GET /api/v1/organizador/eventos/proximos, POST .../buscar, GET .../{id_evento}
13	Calificaciones	Reseñas	Catálogo de	REST - POST

#	Flujo	Quién expone	Quién consume	Backend
	básicas (lote) y reseñas completas de un evento		Eventos	/api/v1/resenas/calificaciones-basicas, GET /api/v1/resenas/evento/{id_evento}
14	Actualización de stock tras compra exitosa	Catálogo de Eventos	Entradas / Inventario	REST - PUT /api/v1/catalogo/eventos/{id_evento}/stock
15	Datos del evento para validar la entrada	Panel Organizador	Check-in	REST - GET /api/v1/panel/eventos/{id_evento}
16	Validación de sesión antes de operaciones protegidas	Auth	Panel Organizador	REST interno - GET /internal/validar-sesion (cabecera Cookie)
17	Cambios de estado y reprogramación de eventos	Panel Organizador	Notificaciones	Broker - exchange panel.evento.notificaciones.v1 (evento_actualizado, evento_reprogramado)
18	Publicación de eventos para validar el ingreso	Panel Organizador	Check-in	Broker (principal) - tópico panel.evento.checkin.v1 + REST de respaldo
19	Actualización de eventos publicados (propuesta de Panel)	Panel Organizador	Catálogo de Eventos	Broker - tópico panel.evento.catalogo.v1 (pendiente de aceptación)
20	Consulta de stock para validar eliminación y disponibilidad	Entradas / Inventario	Panel Organizador	REST - GET /api/v1/entradas/eventos/{id_evento}/stock
21	Identificador del evento para asociar promociones	Panel Organizador	Promociones	Por definir (broker o REST)
22	Recuperación de contraseña y creación de cuenta staff	Auth	Notificaciones	Evento asíncrono - recuperacion_cuenta, cuenta_staff
23	Confirmación de encolamiento del envío del ticket	Notificaciones	Entradas / Inventario	Broker - evento notificaciones.confirmacion_envio (exchange notificaciones.events)

Qué expone cada microservicio

Microservicio	Consumidores de su backend	Flujos
Autenticación (Auth)	Entradas / Inventario, Panel Organizador, Notificaciones	1, 16, 22

Microservicio	Consumidores de su backend	Flujos
Panel Organizador	Entradas / Inventario, Catálogo de Eventos, Check-in, Notificaciones, Promociones	2, 12, 15, 17, 19, 21
Entradas / Inventario	Catálogo de Eventos, Check-in, Notificaciones, Promociones, Panel Organizador	3, 5, 6, 10, 20
Pagos	Entradas / Inventario, Promociones	4, 9
Promociones	Entradas / Inventario	10
Check-in	Reseñas	8
Catálogo de Eventos	Entradas / Inventario (actualización de stock)	14
Reseñas	Catálogo de Eventos	7, 13
Notificaciones	Entradas / Inventario (confirmación de encolamiento del envío)	23

3. Grupo Auth (Autenticación)

Microservicio de identidad de la plataforma. Valida la sesión de los usuarios y entrega el rol vigente, y publica hacia Notificaciones los eventos de recuperación de contraseña y de creación de cuentas de staff. Su alcance es acotado: la introspección de sesión está documentada en los acuerdos de Entradas / Inventario y de Panel Organizador.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Notificaciones <-> Auth	Auth	1.0	recuperacion_cuenta y cuenta_staff (broker)

Detalle de cada contrato

Contrato de backend: Notificaciones <-> Auth (recuperación de contraseña y cuenta staff)

Tema: Notificaciones necesita recibir las solicitudes de recuperación de contraseña y la creación de cuentas de tipo staff para enviar el correo correspondiente. Auth publica los eventos con los datos de contacto y el enlace de restablecimiento, y recibe el resultado del envío.

Versión	1.0 (pendiente confirmar el nombre exacto del evento y el SLA)
Equipo que expone	Auth (dueño del contrato: Diego Peña)
Equipo que consume	Notificaciones (Gabriela Herrera)
Operaciones	Publicar evento recuperacion_cuenta y publicar evento cuenta_staff
Backend	Evento asíncrono publicado en un broker de mensajes (no se usan códigos HTTP)
Datos enviados (recuperacion_cuenta)	Id_usuario, email_destino y url de recuperación
Datos enviados (cuenta_staff)	email_destino
Resultado informado	email_destino, estado_envio (exitoso o error) y error_envio
Basado en	HU7 - Enviar correo de recuperación de contraseña
SLA	Auth publica el evento de inmediato tras procesar la solicitud; confirmación de envío en 5 segundos o menos
Notas	Notificaciones permanece suscrito al evento, genera y envía el correo con la URL recibida, reintenta hasta 3 veces antes de registrar el error y avisa a Auth si el envío fue correcto o no. Quedan por confirmar el nombre de los eventos y el SLA.

4. Grupo Catálogo de Eventos

Microservicio que publica y muestra el catálogo de eventos; es la puerta de entrada del cliente a la plataforma. Consume los datos maestros del Panel Organizador, las calificaciones y reseñas de Reseñas, y el stock que le envía Entradas / Inventario. Sus tres contratos usan la plantilla oficial de Acuerdos de Backend y están marcados como acuerdos formalizados y sincronizados.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Panel Organizador <-> Catálogo de Eventos	Panel Organizador	1.0	Eventos próximos, búsqueda y detalle (REST)
Reseñas <-> Catálogo de Eventos	Reseñas	1.0	Calificaciones básicas y reseñas de un evento (REST)
Entradas / Inventario <-> Catálogo de Eventos	Catálogo de Eventos	3.0	Actualizar stock (REST)

Detalle de cada contrato

Contrato de Backend: Catálogo <-> Panel Organizador

Tema: El Catálogo de Eventos necesita el listado de eventos próximos para la pantalla principal, la búsqueda con filtros y el detalle completo de un evento para sus vistas. Panel Organizador administra la creación y publicación de los eventos y los expone para el consumo del catálogo.

Versión	1.0 (Acuerdo Formalizado y Sincronizado, Sprint 1)
Equipo que expone	Panel Organizador (Integrador Equipo 1)
Equipo que consume	Catálogo de Eventos (Integrador Equipo 7)
Operaciones	1) Solicitar eventos próximos (carga inicial) - 2) Buscar eventos por filtro - 3) Solicitar detalle de evento
Backend	GET /api/v1/organizador/eventos/proximos - POST /api/v1/organizador/eventos/buscar - GET /api/v1/organizador/eventos/{id_evento}
Basado en	Diagrama de secuencia de Catálogo de Eventos
Filtros	Texto libre, categoría, rango de fechas y estado (proximos / pasados)
Notas	Acuerdos firmados: protocolo HTTP REST síncrono y SLA menor a 500 ms en las tres operaciones. Si el estado del evento es pasado o agotado, Catálogo deshabilita el botón de compra. El listado inicial es ligero: solo los datos de la tarjeta.

Contrato de Backend: Catálogo <-> Reseñas

Tema: Catálogo necesita las calificaciones básicas y las reseñas detalladas para mostrar puntajes y opiniones en sus listados y en la vista de detalle de un evento. Reseñas entrega los promedios de estrellas, la cantidad total de valoraciones y el listado de comentarios de cada evento.

Versión	1.0 (Acuerdo Formalizado y Sincronizado, Sprint 1)
Equipo que expone	Reseñas (Integrador Equipo 6)
Equipo que consume	Catálogo de Eventos (Integrador Equipo 7)
Operaciones	1) Solicitar calificaciones básicas (consulta en lote) - 2) Consultar reseñas completas de un evento
Backend	POST /api/v1/resenas/calificaciones-basicas - GET /api/v1/resenas/evento/{id_evento}
Basado en	Diagrama de secuencia de Catálogo de Eventos
Notas	SLA menor a 500 ms. La consulta en lote devuelve solo promedio y total, sin comentarios ni identidades, para no recargar la red en la carga inicial. Ante fallo o timeout, Catálogo degrada a "Sin calificaciones aún" sin interrumpir la compra.

Contrato de Backend: Entradas <-> Catálogo (actualización de stock)

Tema: Tras concretar una compra o emisión de entradas exitosa, Entradas / Inventario envía el stock restante a Catálogo para que su vista de lectura se refresque y los clientes vean la disponibilidad real sin consultar a Entradas en cada momento. Es la persistencia de la consistencia eventual entre la fuente de verdad y la vista de cliente.

Versión	3.0
Equipo que expone	Catálogo de Eventos (Integrador Equipo 7)
Equipo que consume	Entradas / Inventario (Sebastián Fuentes, Scrum Master Equipo 3)
Operación	Actualizar Stock (confirmar_compra_exitosa)
Backend	PUT /api/v1/catalogo/eventos/{id_evento}/stock (id_evento, nuevo_stock, token_sesion)
Basado en	Diagrama de Secuencia "Proceso de Compra" (Distribución Paralela)
Notas	SLA menor a 500 ms. Se descartó el broker (RabbitMQ / Kafka) y se formalizó HTTP REST síncrono. Entradas debe enviar el stock absoluto, ejecuta en paralelo a Check-in y Notificaciones, y Catálogo marca el evento como agotado cuando nuevo_stock es 0.

5. Grupo Check-in

Microservicio de validación de entradas mediante código QR en la puerta del evento. Consume datos de eventos (Panel organizador) y de entradas (Entradas / Inventario), y expone a Reseñas la confirmación de asistencia de un usuario a un evento.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Panel organizador <-> Check-in	Panel organizador	1.0 (borrador)	consultar_evento (REST)
Entradas / Inventario <-	Entradas / Inventario	1.1	consultar_entrada (REST)

Contrato	Equipo que expone	Versión	Operación / backend
> Check-in			
Reseñas <-> Check-in	Reseñas (Equipo 6)	1.1	Verificar asistencia (REST)

Detalle de cada contrato

Contrato de backend: Panel organizador <-> Check-in

Tema: Al validar el ingreso, el staff necesita los datos del evento al que pertenece la entrada (nombre, dirección, fecha y horario) para mostrarlos y confirmar que la entrada corresponde al evento en curso. Check-in no accede a la base de datos de Panel organizador, por lo que consulta los datos por HTTP.

Versión	1.0 (borrador para revisión)
Equipo que expone	Panel organizador
Equipo que consume	Check-in
Operación	consultar_evento — consultar información de evento
Backend	GET /api/v1/panel/eventos/{id_evento}
Basado en	Matriz de dependencias de datos TITECT 2026-2; proyecto TicketU (REST detrás del API Gateway, contratos OpenAPI)
Notas	Nomenclatura de atributos con formato atributo_categoria (id_evento, nombre_evento, fecha_evento). Incluye la sección que define cómo obtener el id_evento desde el QR, ya que Panel organizador no interpreta códigos QR. Sigue en estado de borrador.

Contrato: Entradas / Inventario <-> Check-in

Tema: Al escanear el QR de una entrada, Check-in obtiene el identificador de la entrada (id_ticket) y consulta a Entradas / Inventario los datos asociados para completar la validación, manteniendo la separación de bases de datos entre microservicios.

Versión	1.1
Equipo que expone	Entradas / Inventario
Equipo que consume	Check-in
Operación	consultar_entrada — consultar información de entrada
Backend	GET /api/v1/entradas/{id_ticket}
Basado en	Proceso de validación de entradas mediante QR y dependencia de datos entre microservicios
Notas	Coincide con el contrato equivalente firmado por el equipo de Entradas / Inventario. Conviene mantener una sola versión oficial.

Contrato de backend: Reseñas <-> Check-in

Tema: Antes de habilitar el formulario de reseña, Reseñas necesita confirmar si el usuario asistió al evento. Check-in expone un servicio de solo confirmación (sí / no), sin entregar datos adicionales del asistente.

Versión	1.1
Equipo que expone	Check-in
Equipo que consume	Reseñas (Equipo 6 - Reseñas)
Operación	Verificar asistencia (usuario + evento)
Backend	POST /checkin/asistencia?usuarioId={id}&eventoId={id}
Basado en	Diagrama de secuencia HU-01 (Reseñar evento asistido)
Notas	El equipo de Reseñas documenta el mismo tema con un alcance ampliado que incluye también la HU-05 (adjuntar imagen a la reseña).

6. Grupo Entradas / Inventario

Microservicio dueño del inventario de entradas y del proceso de compra. Valida la identidad del comprador con Auth, cobra con Pagos, valida cupones con Promociones, emite el QR hacia Check-in, sincroniza el stock con Catálogo y notifica la entrada emitida. Su alcance abarca diez acuerdos: los que consume de otros microservicios y los que expone a los demás.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Entradas / Inventario <-> Autenticación (Auth)	Autenticación (Auth)	3.0	Validar sesión (REST interno)
Entradas / Inventario <-> Catálogo de Eventos	Catálogo de Eventos	3.0	Actualizar stock (REST)
Entradas / Inventario <-> Check-in	Check-in	2.1	Enviar QR de acceso (REST)
Entradas / Inventario <-> Pagos	Pagos	2.0 (asíncrono)	Iniciar pago (REST + broker)
Entradas / Inventario <-> Promociones	Promociones	1.0	Validar código (REST)
Catálogo de Eventos <-> Entradas / Inventario	Entradas / Inventario	2.0	Procesar compra (REST)
Entradas <-> Notificaciones	Entradas	3	Entrada emitida (RabbitMQ)
Pagos <-> Entradas / Inventario	Entradas / Inventario	2.1	Gestión de pagos (REST)
Entradas / Inventario <-> Check-in	Entradas / Inventario	1.1	Consultar entrada (REST)
Entradas / Inventario <-> Panel Organizador	Bidireccional	3.0 (unificado)	Sincronización de evento y stock

Detalle de cada contrato

Entradas / Inventario -> Autenticación (Auth): validar sesión

Tema: Antes de comprar, Entradas necesita verificar la identidad del comprador. Following el modelo de Introspección Centralizada de Sesión, Entradas delega por completo la validación del JWT a Auth; Auth verifica vigencia, firma y estado de revocación (tokenVersion) y devuelve un perfil de usuario unificado que queda como única fuente de verdad durante la compra.

Versión	3.0 (Acuerdo Unificado – Introspección Centralizada)
Equipo que expone	Autenticación (Auth)
Equipo que consume	Entradas / Inventario
Operación	Validar Sesión (introspección)
Backend	GET /internal/validar-sesion (endpoint interno)
Basado en	Contrato de integración con Autenticación (Épica de Seguridad, HU-B2, HU-B6)
Notas	Acordado como acuerdo unificado entre los equipos; el endpoint es interno.

Entradas / Inventario -> Catálogo de Eventos: actualizar stock

Tema: Después de una compra o emisión exitosa, Entradas envía el stock actualizado al Catálogo de Eventos para que su vista de lectura se refresque y los clientes vean la disponibilidad real sin consultar a Entradas en cada momento. Persiste la consistencia eventual entre la fuente de verdad (Inventario) y la vista de cliente (Catálogo).

Versión	3.0
Equipo que expone	Catálogo de Eventos
Equipo que consume	Entradas / Inventario
Operación	Actualizar Stock (confirmar_compra_exitosa)
Backend	PUT /api/v1/catalogo/eventos/{id_evento}/stock
Basado en	Diagrama de Secuencia “Proceso de Compra” (Distribución Paralela)
Notas	Contrato firmado por Catálogo de Eventos como proveedor de la actualización de stock. El identificador de versión debe unificarse antes de publicar.

Entradas / Inventario -> Check-in: enviar QR de acceso

Tema: Al finalizar la compra, Entradas entrega a Check-in el ticket consolidado y su código QR para que Check-in registre la entrada como válida y la tenga disponible para el escaneo y la validación rápida en la puerta del evento.

Versión	2.1 (actualización por acuerdos de negocio y persistencia de datos)
Equipo que expone	Check-in
Equipo que consume	Entradas / Inventario
Operación	Enviar QR de Acceso (enviar_qr_generado)

Backend	POST /api/v1/checkin/tickets
Basado en	Acuerdos de negocio del curso TITEC (no referencia HU explícita)
Notas	Complementa el contrato inverso (Check-in consulta la entrada), cerrando el ciclo de la entrada.

Entradas / Inventario -> Pagos: flujo de pago asíncrono

Tema: Como el cobro involucra la interacción del usuario y validaciones bancarias que exceden los tiempos de una petición HTTP normal, Entradas inicia la transacción por REST, recibe un estado PENDIENTE y luego espera pasivamente un evento en el broker (RabbitMQ) para consolidar la venta y emitir la entrada, o liberar el stock reservado.

Versión	2.0 (actualización a flujo asíncrono)
Equipo que expone	Pagos
Equipo que consume	Entradas / Inventario
Operaciones	Síncrona: iniciar pago (iniciar_pago), Asíncrona: evento de resultado del pago
Backend	POST /api/v1/pagos/transacciones (inicio) y consulta de estado GET /api/v1/pagos/{id_pago}; broker RabbitMQ para el resultado
Basado en	Revisión de viabilidad técnica y latencia de la pasarela externa
Notas	Es la evolución del flujo síncrono que el equipo de Pagos documenta en su versión 1.0.

Entradas / Inventario -> Promociones: validación de cupón en el checkout

Tema: En el checkout, el usuario puede aplicar un código promocional. Promociones evalúa si el código cumple límites de uso, vigencia y cantidad, y devuelve el porcentaje de descuento aplicable y el evento asociado para que el cobro sea procesado.

Versión	1.1
Equipo que expone	Promociones
Equipo que consume	Entradas
Operación	Validación de código (el documento conserva el nombre de operación sin completar)
Backend	POST /promociones/validar/{nombre_codigo}
Basado en	HU-03, HU-05, HU-07, HU-08
Notas	Coincide con el contrato equivalente que firma el equipo de Promociones (validación de cupón en el checkout).

Catálogo de Eventos -> Entradas / Inventario: procesar compra

Tema: El Catálogo de Eventos solicita a Entradas / Inventario procesar la compra de las entradas seleccionadas por el cliente. Entradas gestiona la reserva, procesa la orden y devuelve la confirmación de la transacción junto con la disponibilidad restante.

Versión	2.0
Equipo que expone	Entradas / Inventario
Equipo que consume	Catálogo de Eventos
Operación	Procesar compra de entradas
Backend	POST /api/v1/entradas/procesar-compra
Basado en	Diagrama de secuencia de Catálogo de Eventos
Notas	Es la contraparte del contrato de actualización de stock: la compra entra por Catálogo y el stock actualizado vuelve a él.

Entradas -> Notificaciones: entrada emitida por correo

Tema: Una vez emitida la entrada con su código QR, Notificaciones necesita los datos para enviar el ticket al comprador por correo. La comunicación es totalmente asíncrona: Entradas publica el evento entradas_emitidas en el exchange entradas.events y Notificaciones lo consume.

Versión	3
Equipo que expone	Entradas
Equipo que consume	Notificaciones
Operación	Enviar entrada emitida por correo (evento de dominio)
Backend	RabbitMQ – exchange entradas.events, evento entradas_emitidas; confirmación en notificaciones.confirmacion_envio
Basado en	HU1 – Recibir notificación de compra
Notas	Único contrato puramente asíncrono del conjunto; no aplica endpoint REST.

Pagos <-> Entradas / Inventario: respuesta de Pagos al flujo asíncrono

Tema: Documento mediante el cual Pagos responde al flujo asíncrono propuesto por Entradas v2.0: crea el pago en estado PENDIENTE directamente en Stripe y devuelve la url_checkout donde el comprador ingresa su tarjeta. También permite consultar el estado y anular o reembolsar el pago si la reserva vence o se queda sin stock.

Versión	2.1 (API v1) – 28/09/2026
Equipo que expone	Pagos
Equipo que consume	Entradas / Inventario
Operaciones	Crear intención de pago, Consultar estado del pago, Anular o reembolsar un pago
Backend	POST /api/v1/pagos/transacciones con Authorization: Bearer <JWT de Auth>
Basado en	Contrato Entradas -> Pagos v2.0 (Sebastián Fuentes); HU 1.3, 1.6, 3.1 y 3.3; ítems de integración INT-02 e INT-03; tabla de dependencias TITEC 2026-2; Swagger con la definición OpenAPI de la API
Notas	El cobro no se resuelve en una sola petición. La publicación de eventos en el broker queda declarada

	como pendiente (sección 12 del documento).
--	--

Entradas / Inventario <-> Check-in: consultar información de entrada

Tema: Acuerdo de consulta que Check-in realiza a Entradas / Inventario al escanear el QR (id_ticket) para obtener los datos de la entrada y continuar con la validación.

Versión	1.1
Equipo que expone	Entradas / Inventario
Equipo que consume	Check-in
Operación	consultar_entrada
Backend	GET /api/v1/entradas/{id_ticket}
Basado en	Proceso de validación de entradas mediante QR y dependencia de datos entre microservicios
Notas	Reproduce exactamente el contrato Entradas / Inventario <-> Check-in (versión 1.1) firmado por el equipo de Check-in.

Contrato unificado Entradas / Inventario <-> Panel Organizador

Tema: Acuerdo unificado para recibir del Panel Organizador los datos maestros de los eventos, sincronizar su ciclo de vida (creación, edición, publicación, cancelación y eliminación) y reportar los cambios críticos de aforo, exponiendo la disponibilidad real del stock. Es el contrato más completo del proyecto en cuanto a glosario y ciclo de vida.

Versión	3.0 (Acuerdo Unificado – Perspectiva Entradas)
Comunicación	Bidireccional: broker RabbitMQ y HTTP REST
Equipo principal	Entradas / Inventario (documentador)
Equipo contraparte	Panel Organizador
Operaciones	Recepción de eventos de ciclo de vida del evento, Consulta de datos de evento, Publicación de stock disponible
Backend	Tópico a escuchar: panel.evento.entradas.v1, Tópico de publicación: entradas.evento.stock.v1, REST: GET /api/v1/panel/eventos/{id_evento} y GET /api/v1/entradas/eventos/{id_evento}/stock
Glosario	tipo_entrada: PAGADA, GRATUITA, estado_gestion: BORRADOR, PUBLICADO, FINALIZADO, CANCELADO
Notas	Adopta el glosario propuesto por Panel para garantizar consistencia entre servicios.

7. Grupo Notificaciones

Microservicio de mensajería de la plataforma. No expone datos de negocio: recibe eventos de otros equipos, identifica a los usuarios afectados, encola y envía los correos (ticket de compra, cambios o cancelación de eventos, recuperación de contraseña y cuentas de tipo staff) e informa al organizador el resultado. Mantiene un contrato con cada equipo que le publica información.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Entradas <-> Notificaciones	Entradas / Inventario	3.0	Entrada emitida con QR (broker) + confirmación de encolamiento
Panel Organizador <-> Notificaciones	Panel Organizador	1.0	evento_actualizado y evento_reprogramado (broker)
Notificaciones <-> Auth	Auth	3.0	recuperacion_cuenta y cuenta_staff (broker)

Detalle de cada contrato

Contrato de backend: Entradas <-> Notificaciones (ticket de compra por correo)

Tema: Cuando una entrada queda emitida, el comprador debe recibir su ticket por correo. Entradas publica el evento con los datos del comprador, los del evento y el contenido del QR; Notificaciones lo consume, encola el envío, confirma la recepción a Entradas y reintenta hasta 3 veces ante cualquier fallo.

Versión	3.0 (acordado en su totalidad, sin pendientes)
Equipo que expone	Entradas / Inventario (Sebastián Fuentes)
Equipo que consume	Notificaciones (Gabriela Herrera)
Operación	Enviar la entrada emitida por correo
Backend	RabbitMQ: exchange entradas.events con el evento entradas_emitidas; la recepción se confirma con el evento notificaciones.confirmacion_envio del exchange notificaciones.events
Datos recibidos	correo_comprador, nombre_comprador, id_usuario, nombre_evento, id_evento, fecha_evento, hora_evento y qr_data (URL del QR o contenido en Base64)
Confirmación devuelta	estado (en cola o programado) y mensaje
SLA	Menor a 200 ms, considerando solo la recepción y el encolamiento de la solicitud
Basado en	HU1 - Recibir notificación de compra
Notas	El envío se procesa de forma asíncrona. Notificaciones guarda en su base de datos los errores por datos faltantes o correo no enviado, y la confirmación del envío exitoso o fallido del correo es opcional. Cualquier cambio en la estructura del evento genera una nueva versión del contrato.

Contrato de backend: Panel Organizador <-> Notificaciones (cambios y cancelaciones de evento)

Tema: Cuando un organizador cancela o modifica un evento, los usuarios con entradas activas deben ser informados. Panel publica el evento con el nuevo estado; Notificaciones identifica a los usuarios afectados, genera y envía las notificaciones e informa el resultado directamente al organizador.

Versión	1.0 (pendiente confirmar el SLA de tiempo de respuesta)
Equipo que expone	Panel Organizador (María Baxmann)
Equipo que consume	Notificaciones (Gabriela Herrera)
Operaciones	Publicar el evento evento_actualizado y publicar el evento evento_reprogramado
Backend	Evento asíncrono por broker, sin códigos HTTP; exchange panel.evento.notificaciones.v1
Mensaje evento_actualizado	id_evento, id_usuario y nuevo_estado (borrador, cancelado, finalizado o publicado)
Mensaje evento_reprogramado	id_evento, id_usuario, nuevo_estado, fecha_cambio y hora_cambio
Resultado informado	id_evento, id_usuario, nuevo_estado (exitoso o error), usuarios_notificados, usuarios_faltantes y fecha_envio
Errores	Sin códigos HTTP; hasta 3 intentos antes de registrar el error
SLA	Publicación inmediata tras el cambio, con 5 segundos o menos hasta la confirmación de las notificaciones
Basado en	HU6 - Recibir notificación por cancelación o cambios de evento
Notas	Notificaciones obtiene por sí misma los usuarios con entradas activas, ya que Panel no envía listas de asistentes. El resultado se informa al organizador mediante el sistema de notificaciones de Notificaciones, sin comunicación de retorno a Panel. Un cambio incompatible requiere un exchange nuevo.

Contrato de backend: Notificaciones <-> Auth (recuperación de contraseña y cuenta staff)

Tema: Notificaciones necesita los datos de las solicitudes de recuperación de contraseña y de la creación de cuentas de tipo staff para enviar el correo correspondiente. Auth publica los eventos con el contacto del usuario y el enlace de restablecimiento.

Versión	3.0 (pendiente confirmar el SLA de tiempo de respuesta)
Equipo que expone	Auth (Diego Peña)
Equipo que consume	Notificaciones (Gabriela Herrera)
Operaciones	Publicar el evento recuperacion_cuenta y publicar el evento cuenta_staff
Backend	Evento asíncrono por broker de mensajes (no se usan códigos HTTP)
Evento recuperacion_cuenta	id_usuario, email_destino y url de recuperación
Evento cuenta_staff	email_destino y url de configuración de la cuenta temporal
Resultado informado	email_destino, estado_envio (exitoso o error) y error_envio

Errores	Sin códigos HTTP; hasta 3 intentos de envío antes de registrar el error
SLA	Auth publica el evento de inmediato tras procesar la solicitud, con 5 segundos o menos hasta recibir la confirmación de envío
Basado en	HU7 - Enviar correo de recuperación de contraseña
Notas	Notificaciones permanece suscrito a ambos eventos, genera el correo con la URL recibida y avisa a Auth si el envío fue correcto o no. Los cambios que rompan la compatibilidad requieren un período de transición acordado.

8. Grupo Pagos

Microservicio de cobro. Expone la creación, consulta, anulación y reembolso de pagos, y es proveedor tanto de Entradas / Inventario como de Promociones. Delega el cobro de la tarjeta directamente a la pasarela externa (Stripe) mediante una url_checkout.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Entradas / Inventario <-> Pagos	Pagos	1.0	Crear / consultar / anular pago (REST)
Promociones <-> Pagos	Pagos	1.0	Consultar pago aprobado (REST)

Detalle de cada contrato

Contrato de backend: Entradas <-> Pagos

Tema: Entradas necesita cobrar la orden de compra de una reserva y saber si el pago fue aprobado para confirmar o liberar las entradas. Pagos recibe la orden (cantidad de entradas y total) y devuelve el id del pago, su estado y la fecha de creación; además permite anular o reembolsar cuando se libera la reserva.

Versión	1.0
Equipo que expone	Pagos
Equipo que consume	Entradas / Inventario
Operaciones	Crear pago, Consultar estado del pago, Anular o reembolsar el pago
Backend	POST /api/v1/pagos/transacciones
Autenticación	JWT emitido por Auth en la cabecera Authorization: Bearer <token>
Basado en	HU 1.3, HU 1.6, HU 3.1 y HU 3.3, Tabla de dependencias TITEC (Pagos envía a Entradas: fecha de creación de la compra, estado del pago e id del pago)
Notas	Es la versión síncrona del flujo; el equipo de Entradas / Inventario maneja la v2.0 asíncrona y la respuesta de Pagos v2.1. Conviene usar la v2.1 como

	versión oficial única.
--	------------------------

Contrato de backend: Promociones <-> Pagos

Tema: Promociones necesita confirmar que una compra con descuento fue efectivamente pagada para rebajar la cantidad de códigos restantes en su base de datos; para ello consulta el id del pago aprobado de la reserva o del usuario.

Versión	1.0
Equipo que expone	Pagos
Equipo que consume	Promociones
Operación	Consultar pago aprobado
Backend	GET /api/v1/pagos?id_reserva={id} &estado=APROBADO&pagina=1&limite=20
Basado en	HU 3.4, Tabla de dependencias TITEC (Pagos envía a Promociones: id del pago)
Notas	El equipo de Promociones plantea la alternativa asíncrona para el mismo objetivo (tópico pago.completado); ambas formas conviven hoy.

9. Grupo Panel Organizador

Microservicio administrativo de los eventos: es la fuente de los datos generales, del estado de gestión y de la disponibilidad que consumen el resto de microservicios. Publica los cambios por broker, atiende las consultas REST de respaldo y delega en Auth la validación de sesión antes de sus operaciones protegidas. Mantiene un contrato con cada microservicio con el que se comunica.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Auth <-> Panel Organizador	Auth	1.1	Validar sesión (REST interno)
Panel Organizador <-> Catálogo de Eventos	Panel Organizador	1.1	Próximos, búsqueda y detalle (REST) + broker propuesto
Panel Organizador <-> Check-in	Panel Organizador	1.2	Publicación de eventos (broker) + consulta de respaldo (REST)
Panel Organizador <-> Entradas / Inventario	Panel Organizador	1.2	Datos del evento (broker y REST) + consulta de stock
Panel Organizador <-> Notificaciones	Panel Organizador	1.3	evento_actualizado y evento_reprogramado (broker)
Panel Organizador <-> Promociones	Panel Organizador	1.1	id_evento del evento disponible (mecanismo por definir)

Detalle de cada contrato

Contrato de backend: Auth <-> Panel Organizador

Tema: Antes de ejecutar operaciones protegidas, Panel Organizador necesita confirmar la identidad y la vigencia de la sesión del organizador. Auth valida la autenticación y entrega el perfil y el rol vigente; Panel aplica sus propias reglas de autorización sobre los eventos, incluida la validación de que el organizador es dueño del evento.

Versión	1.1 (propuestas de SLA y reintentos pendientes de confirmación)
Equipo que expone	Auth
Equipo que consume	Panel Organizador
Operación	Validación de sesión (introspección)
Backend	GET /internal/validar-sesion, reenviando la cabecera Cookie con el token
Datos recibidos	valido y usuario (id_usuario, nombre_completo, rut, correo_electronico, rol)
Errores	401 detiene la operación, 403 la bloquea, 500 la aborta y 503 aplica Fail-Secure
Responsabilidades	Validación de JWT e identidad vigente de Auth; autorización sobre operaciones y propiedad del evento de Panel
Propuestas de Panel	SLA menor a 300 ms, timeout de 1 segundo, 1 reintento ante indisponibilidad y dirección del servicio en la variable AUTH_SERVICE_URL
Notas	Panel no decodifica ni valida el JWT localmente y no usa como identidad el id_usuario enviado por el Frontend. No hay reintentos ante 401 o 403. Auth debe confirmar el SLA, el timeout, la política de reintentos y la URL del ambiente de integración.

Contrato de backend: Panel Organizador <-> Catálogo de Eventos

Tema: Catálogo necesita los datos públicos de los eventos para su pantalla principal, sus filtros y la vista de detalle. Panel los expone con tres consultas REST y propone además publicar los cambios por broker para que el catálogo se actualice sin consultar en cada momento.

Versión	1.1 (pendientes de confirmación por Catálogo)
Equipo que expone	Panel Organizador
Equipo que consume	Catálogo de Eventos
Operaciones	Solicitar eventos próximos, buscar eventos por filtro, consultar detalle de evento
Backend	GET /api/v1/organizador/eventos/proximos, POST /api/v1/organizador/eventos/buscar y GET /api/v1/organizador/eventos/{id_evento}; broker propuesto con tópico panel.evento.catalogo.v1
Datos del evento	id_evento, nombre_evento, descripcion, direccion_evento, fecha_evento, hora_evento, imagen, categoria, precio, tipo_entrada, estado_gestion y estado_disponibilidad

Estados	estado_gestion (BORRADOR, PUBLICADO, FINALIZADO, CANCELADO) y estado_disponibilidad (DISPONIBLE, AGOTADO, PASADO) como conceptos separados
SLA	Menor a 500 ms en las consultas REST
Notas	Panel no envía el campo stock, que pertenece a Entradas / Inventario, y marca AGOTADO a partir de la información de disponibilidad que recibe. Catálogo debe aceptar los nombres de campo y los valores de estado, y confirmar si admite el broker para las actualizaciones.

Contrato de backend: Panel Organizador <-> Check-in

Tema: El staff necesita identificar, ubicar y verificar la vigencia del evento al que pertenece la entrada escaneada. Panel entrega el estado completo del evento por broker como mecanismo principal y atiende una consulta REST de respaldo cuando Check-in no lo tiene en su copia local. Quedan fuera de este contrato el contenido del QR, la obtención del id_evento desde la entrada y la lista de asistencia.

Versión	1.2 (Check-in debe confirmar diez puntos pendientes)
Equipo que expone	Panel Organizador (Mariajosé Baxmann)
Equipo que consume	Check-in
Operaciones	Publicación del evento al cambiar su estado o sus datos (principal) y consulta de evento por identificador (respaldo)
Backend	Broker RabbitMQ con tópico panel.evento.checkin.v1; consulta de respaldo GET /api/v1/panel/eventos/{id_evento}
Datos del evento	id_evento, nombre_evento, direccion_evento, fecha_evento, hora_evento, hora_fin_evento, estado_gestion y, si Check-in lo necesita, cantidad_entradas como aforo
Disparadores de publicación	PUBLICADO, ACTUALIZADO, CANCELADO y FINALIZADO, con fecha_cambio en ISO 8601
Reglas de horario	hora_evento es la hora de inicio y hora_fin_evento la de término; si hora_fin_evento es menor o igual que hora_evento, el evento termina al día siguiente, en zona horaria America/Santiago
Errores REST	400 ID_EVENTO_INVALIDO, 401 SESION_INVALIDA, 403 ROL_NO_AUTORIZADO, 404 EVENTO_NO_ENCONTRADO, 500 ERROR_INTERNO y 503 SERVICIO_NO_DISPONIBLE, con la estructura codigo_http, error y mensaje definida por Auth
SLA	Menor a 300 ms en la consulta de respaldo, incluyendo la validación de sesión con Auth
Notas	Cola durable, confirmación del mensaje después de guardarlo en la copia local y hasta 3 reintentos antes de registrar el error; Check-in descarta los mensajes cuya fecha_cambio no sea más reciente. Panel valida la sesión con Auth en cada consulta REST y aplica Fail-Secure con 503 si Auth no responde; los mensajes del

	broker no usan cookie de sesión.
--	----------------------------------

Contrato de backend: Panel Organizador <-> Entradas / Inventario

Tema: Entradas / Inventario necesita los datos generales del evento y su estado de gestión para operar, y Panel necesita conocer el stock para determinar la disponibilidad y validar la eliminación de un evento. El intercambio se apoya en el broker como mecanismo principal y en una consulta REST de respaldo.

Versión	1.2 (pendientes de confirmación de Entradas)
Equipo que expone	Panel Organizador
Equipo que consume	Entradas / Inventario
Operaciones	Consulta de información del evento, sincronización de cambios por broker, consulta de stock por parte de Panel y notificación de evento agotado
Backend	GET /api/v1/panel/eventos/{id_evento} (SLA menor a 300 ms); tópico panel.evento.entradas.v1 para creación, edición, publicación, cancelación y eliminación; GET /api/v1/entradas/eventos/{id_evento}/stock para consultar el stock; tópico entradas.evento.stock.v1 cuando el stock llega a 0
Datos compartidos	id_evento, id_usuario, nombre_evento, fecha_evento, hora_evento, cantidad_entradas, tipo_entrada y estado_gestion
Nomenclatura propuesta	tipo_entrada PAGADA o GRATUITA y estado_gestion BORRADOR, PUBLICADO, FINALIZADO o CANCELADO
Validación de eliminación	Si el stock es igual a cantidad_entradas, Panel puede eliminar el evento; si es menor, no lo elimina y fija estado_gestion = CANCELADO
Evento agotado	Con stock en 0, Panel marca la disponibilidad como AGOTADO y la comunica a los microservicios que la consumen
Notas	La disponibilidad es un concepto independiente del estado de gestión. Las devoluciones y reembolsos quedan fuera de este contrato y corresponden a Pagos. Entradas debe confirmar el uso del broker, los valores de tipo_entrada y estado_gestion, la consulta de stock y su comportamiento tras una cancelación.

Contrato de backend: Panel Organizador <-> Notificaciones

Tema: Cuando cambia el estado de un evento o se reprograma, los usuarios con entradas activas deben recibir una notificación. Panel publica el cambio por broker; Notificaciones identifica a los usuarios afectados a partir del id_evento, envía los mensajes e informa el resultado directamente al organizador.

Versión	1.3 (con historial documentado desde la v1.0)
Equipo que expone	Panel Organizador (Mariajosé Baxmann)
Equipo que consume	Notificaciones (Gabriela Herrera)

Operaciones	Publicar mensaje evento_actualizado y publicar mensaje evento_reprogramado
Backend	Exchange panel.evento.notificaciones.v1 (broker, sin códigos HTTP)
Mensaje evento_actualizado	id_evento, id_usuario y nuevo_estado (BORRADOR, PUBLICADO, FINALIZADO o CANCELADO)
Mensaje evento_reprogramado	id_evento, id_usuario, nuevo_estado, fecha_cambio y hora_cambio
Resultado del envío	id_evento, id_usuario, nuevo_estado (exitoso o error), usuarios_notificados, usuarios_faltantes y fecha_envio
SLA	Panel publica el mensaje en 5 segundos o menos después de guardar el cambio; el envío a usuarios y el informe al organizador dependen de Notificaciones
Notas	Notificaciones obtiene por su cuenta los usuarios con entradas activas, ya que Panel no envía listas de asistentes. Hasta 3 intentos ante error de procesamiento o envío. No existe comunicación de Notificaciones hacia Panel: el resultado se informa al organizador por el sistema de notificaciones de Notificaciones. Un cambio incompatible genera un nuevo exchange, por ejemplo panel.evento.notificaciones.v2.

Contrato de backend: Panel Organizador <-> Promociones

Tema: Para que Promociones pueda asociar sus promociones a un evento, Panel comparte el identificador del evento cuando este queda disponible. La creación y gestión de las promociones corresponde exclusivamente a Promociones.

Versión	1.1 (contrato en estado de propuesta, con el mecanismo sin definir)
Equipo que expone	Panel Organizador
Equipo que consume	Promociones
Operación	Compartir el identificador del evento disponible
Backend	Por definir (broker o REST); sin tópico ni endpoint asignado
Dato compartido	id_evento
Responsabilidad de Panel	Entregar el identificador del evento; no genera códigos promocionales, descuentos, vigencias, cantidades ni precios resultantes
Notas	Promociones debe confirmar el mecanismo de comunicación, el tópico o endpoint, el momento exacto en que necesita el identificador y si requiere datos adicionales del evento. Por ahora no se define comunicación de Promociones hacia Panel.

10. Grupo Promociones

Microservicio de códigos promocionales y cupones. Valida los códigos en el checkout devolviendo el descuento aplicable y descuenta los códigos usados cuando el pago se completa.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Entradas <-> Promociones	Promociones	1.0	Validar código en checkout (REST)
Promociones <-> Pagos	Pagos	1.0	Consumo de cupón (evento asíncrono)

Detalle de cada contrato

Contrato de backend: Entradas <-> Promociones (validación en checkout)

Tema: El checkout necesita validar que un código promocional esté vigente y calcular el descuento. Promociones recibe cantidad de entradas, precio y usuario, y devuelve el porcentaje de descuento aplicable y el evento asociado para que el cobro sea procesado.

Versión	1.0
Equipo que expone	Promociones
Equipo que consume	Entradas
Operación	Validación de código (el documento conserva el nombre de operación sin completar)
Backend	POST /promociones/validar/{nombre_codigo}
Basado en	HU-03, HU-05, HU-07, HU-08
Notas	Coincide con el contrato equivalente firmado por Entradas / Inventario. Queda pendiente confirmar el nombre de la operación, hoy en texto de ejemplo.

Contrato de backend: Promociones <-> Pagos (consumo de cupón)

Tema: Promociones necesita verificar que la entrada con descuento fue comprada para rebajar la cantidad de códigos restantes. Se define que Pagos transmita un evento notificando el cobro exitoso de la transacción, incluyendo el identificador del código de descuento si se aplicó uno.

Versión	1.0
Equipo que expone	Pagos
Equipo que consume	Promociones
Operación	Consumo de cupón por pago completado (nombre de operación sin definir en el documento)
Backend	Evento asíncrono (broker) – tópico pago.completado
Basado en	HU-06
Notas	Contrato en estado de plantilla: mantiene secciones de ejemplo sin completar (nombre de la operación, request/response y SLA).

11. Grupo Reseñas

Microservicio de reseñas y calificación promedio. Consume del Catálogo los datos mínimos del evento y de Check-in la confirmación de asistencia; expone al Catálogo el promedio y el listado de reseñas ya calculado y ordenado.

Contratos del grupo

Contrato	Equipo que expone	Versión	Operación / backend
Reseñas <-> Check-in	Check-in	1.1	Verificar asistencia (REST)
Reseñas <-> Catálogo de Eventos	Catálogo de Eventos	1.0	Obtener datos del evento (REST)
Catálogo de Eventos <-> Reseñas	Reseñas	1.0	Obtener reseñas y promedio (REST)

Detalle de cada contrato

Contrato de backend: Reseñas <-> Check-in

Tema: Reseñas habilita el formulario de reseña solo si el usuario asistió al evento. Check-in expone un servicio de solo confirmación por usuario y evento, sin entregar datos adicionales del asistente.

Versión	1.1
Equipo que expone	Check-in
Equipo que consume	Reseñas
Operación	Verificar asistencia
Backend	POST /checkin/asistencia?usuarioId={id}&eventoId={id}
Basado en	Diagrama de secuencia HU-01 (Reseñar evento asistido) y HU-05 (Adjuntar imagen a la reseña)
Notas	Equivalente al contrato Reseñas <-> Check-in (versión 1.1) con el alcance ampliado a la HU-05. Incluye reglas de uso, versionado y responsables del contrato.

Contrato de backend: Reseñas <-> Catálogo de Eventos

Tema: Reseñas necesita datos básicos del evento (por ejemplo, nombre y fecha) para mostrarlos junto a la reseña publicada y para validar que el evento exista antes de habilitar el flujo. Catálogo los entrega de forma acotada, sin información interna de gestión salvo acuerdo explícito.

Versión	1.0
Equipo que expone	Catálogo de Eventos
Equipo que consume	Reseñas (Equipo 6 - Reseñas)
Operación	Obtener datos del evento
Backend	GET /eventos/{eventoId} (propuesta REST, pendiente de confirmar)
Basado en	HU-01 (Reseñar evento asistido) y HU-02 (Adjuntar imagen a la reseña)

Notas	Documento en Markdown, con reglas de uso, versionado, owners y lista de pendientes a acordar con Catálogo.
--------------	--

Contrato de backend: Catálogo de Eventos <-> Reseñas

Tema: En el detalle de un evento, Catálogo necesita mostrar la calificación promedio y el listado de reseñas con comentario, ordenado de más reciente a menos reciente. Reseñas entrega el promedio ya calculado y el listado ya ordenado para que Catálogo no acceda a los datos crudos ni recalcule.

Versión	1.0
Equipo que expone	Reseñas
Equipo que consume	Catálogo de Eventos
Operación	Obtener reseñas y promedio de un evento
Backend	GET /resenas/evento/{eventoId} (propuesta REST, pendiente de confirmar)
Basado en	Diagrama de secuencia HU-02 (Ver reseñas y calificación promedio de un evento)
Notas	Diseño definido: sin reseñas se retorna promedio null y resenas vacío, nunca un error, para mostrar estado vacío explícito. SLA propuesto menor a 300 ms por bloquear la UI.

12. Notas transversales y puntos a unificar

Observaciones que surgen sobre el conjunto de los contratos revisados, útiles para la coordinación entre grupos.

Tema	Detalle
Acuerdos documentados por los dos lados	Hay cinco pares de contratos que aparecen firmados por ambos equipos: Entradas / Inventario <-> Check-in (v1.1), Entradas <-> Promociones (v1.0), Reseñas <-> Check-in (v1.1), Entradas <-> Catálogo (v3.0) y Catálogo <-> Reseñas. En todos ellos conviene que exista una única versión oficial, acordada por las dos partes.
Rutas distintas para el mismo contrato con Reseñas	El contrato de Catálogo de Eventos define GET /api/v1/resenas/evento/{id_evento} y además la consulta en lote POST /api/v1/resenas/calificaciones-basicas; el equipo de Reseñas propone GET /resenas/evento/{eventoId}. Hay que unificar la ruta, el nombre del parámetro y decidir si la consulta en lote queda dentro del mismo contrato.
Alcance del contrato con Panel Organizador	Catálogo consulta los eventos directamente a Panel Organizador, mientras Entradas / Inventario los recibe por broker. Conviene evaluar si Catálogo debe consumir el tópico ya publicado por Entradas en lugar de consultar por su cuenta.
Versiones en conflicto del cobro	El equipo de Pagos documenta el flujo de cobro en su versión 1.0 (síncrono), mientras Entradas / Inventario maneja la v2.0 asíncrona y la respuesta de Pagos v2.1. Debería prevalecer la v2.1 como versión vigente.
Documentos en estado plantilla	Los dos contratos del equipo de Promociones y el de Entradas <-> Promociones mantienen el nombre de la operación como texto de ejemplo ("[Nombre de la operación, ej. ...]") y request/response sin concretar. En contraste, los tres contratos de Catálogo de Eventos ya están formalizados y firmados.
Identificador de versión inconsistente en Catálogo	En el contrato Entradas <-> Catálogo la cabecera declara la versión 3.0, mientras una de las copias se identifica como v4. Ambas coinciden en el endpoint PUT de actualización de stock; conviene alinear el identificador de versión.
Protocolos de integración en uso	REST síncrono para la mayoría de las consultas; eventos asíncronos en tres casos: Entradas -> Notificaciones (RabbitMQ), Entradas / Panel Organizador (RabbitMQ + REST) y Promociones <- Pagos (tópico pago.completado). El contrato de Catálogo con Entradas descartó expresamente el broker en favor de REST síncrono.
Contrato de sesión de Auth documentado en tres equipos	El equipo de Auth solo publica el contrato con Notificaciones. La introspección de sesión aparece en el acuerdo de Entradas / Inventario (v3.0) y en el de Panel Organizador (v1.1), ambos sobre GET /internal/validar-sesion. Conviene un único contrato de Auth, con los valores de SLA, timeout y reintentos que Panel propone pendientes de confirmación.

Panel Organizador propone broker donde el otro equipo ya fijó REST	Panel propone publicar en panel.evento.catalogo.v1, mientras Catálogo ya formalizó REST síncrono y descartó expresamente el broker. Con Check-in, en cambio, se acordó el modelo mixto: broker principal y consulta REST de respaldo. Hay que decidir un único criterio según el caso de uso.
Nomenclatura de estados y tipo de entrada	Panel propone tipo_entrada PAGADA o GRATUITA y estado_gestion (BORRADOR, PUBLICADO, FINALIZADO, CANCELADO); Entradas usa pago y gratis, además de estado_evento con valores como ACTIVO; Catálogo espera valores en minúscula como pasado y agotado. Deben unificarse el nombre del campo y los valores admitidos.
Consulta de stock propuesta por Panel	Panel necesita conocer el stock para validar la eliminación física de un evento y para marcar la disponibilidad como AGOTADO, mediante GET /api/v1/entradas/eventos/{id_evento}/stock y la notificación en entradas.evento.stock.v1. Entradas debe confirmar ese endpoint, ya que su contrato solo documenta la actualización de stock con PUT.
Los tres contratos de Notificaciones están firmados también por el otro equipo	El equipo de Notificaciones mantiene copias del acuerdo con Entradas / Inventario (v3.0), con Panel Organizador (v1.0) y con Auth (v3.0). La copia de Panel va en v1.0 mientras la de Entradas va en v1.3, y la de Auth en v3.0 frente a la v1.0 del propio Auth. Conviene una sola versión por acuerdo, alojada por el equipo que expone el evento.
Campos que difieren entre las dos copias del evento de recuperación de cuenta	La copia de Auth envía solo email_destino en cuenta_staff, mientras la de Notificaciones añade la url de configuración de la cuenta temporal. Además el identificador aparece como Id_usuario en una y como id_usuario en la otra, y email_destino está declarado como integer en ambas. Se debe fijar un solo nombre de campo, el tipo correcto y la presencia de la url.
Pendientes comunes en los documentos	Confirmación de protocolo exacto, autenticación, códigos de error, SLA de respuesta, nombres de endpoints y criterios de versionado y cambios con período de transición para los breaking changes.

Documento de trabajo: consolida el tema de cada acuerdo de backend y cómo quedó definido.